

OpsFlow 核心引擎

从模板到执行 — 完整流程详解

2026-06-03

Generated: 2026-06-03 | Source: backend/opsflow/doc/

1. FlowEngine 类

[FlowEngine] 位于 [opsflow/core/flow_engine.py]，是流程执行的核心入口。所有执行操作通过 [bamboo_engine.api] 委托到 [BambooDjangoRuntime]。

1.1 构造

```
class FlowEngine:
    def __init__(self, execution):
        self.execution = execution # FlowExecution 实例
        self.template = execution.template
```

1.2 API 总览

方法	用途	底层调用	影响的状态
<code>`start(sync=False)`</code>	启动执行，派发到 Celery	<code>`execute_pipeline_task.delay(id)`</code>	pending → running
<code>`pause()`</code>	暂停执行	<code>`pipeline_api.pause_pipeline()`</code>	running → paused
<code>`resume()`</code>	恢复执行	<code>`pipeline_api.resume_pipeline()`</code>	paused → running

方法	用途	底层调用	影响的状态
<code>`retry(node_id)`</code>	重试指定失败节点	<code>`pipeline_api.retry_node()`</code>	<code>node_status[node] = "retrying"</code>
<code>`retry_subprocess(node_id)`</code>	重试子流程节点	<code>`pipeline_api.retry_subprocess()`</code>	<code>node_status[node] = "retrying"</code>
<code>`skip(node_id)`</code>	跳过指定失败节点	<code>`pipeline_api.skip_node()`</code>	<code>node_status[node] = "skipped"</code>
<code>`force_fail(node_id)`</code>	强制失败指定节点	<code>`pipeline_api.forced_fail_activity()`</code>	<code>node_status[node] = "failed"</code>
<code>`cancel()`</code>	取消终止执行	<code>`pipeline_api.revoke_pipeline()`</code>	<code>running → cancelled</code>
<code>`run()`</code>	执行 Pipeline (Celery 回调)	<code>`pipeline_api.run_pipeline()`</code>	构建 + 调度 Pipeline
<code>`rollback_failed_nodes()`</code>	失败节点回滚	遍历 + <code>`_trigger_plugin_rollback()`</code>	无状态变更

1.3 run() 方法完整流程

```

def run(self):
    # 1. 获取创建时冻结的 template_snapshot (实现执行隔离)
    frozen = self.execution.template_snapshot or {}
    frozen_tree = frozen.get('pipeline_tree')

    # 2. 执行前校验 pipeline_tree
    if frozen_tree:
        validation = validate_pipeline(frozen_tree)
        if not validation['valid']:
            self._fail_execution(errors) # 提前失败
            return

    # 3. 构建 bamboo-engine Pipeline Tree
    pipeline, id_map = build_bamboo_pipeline(
        self.template,
        pipeline_tree=frozen_tree,
        target_hosts=frozen.get('target_hosts'),
        global_vars=frozen.get('global_vars'),
        execution_id=self.execution.id,
        excluded_nodes=self.execution.excluded_nodes,
    )

    # 4. 空 pipeline 检测
    if len(acts) == 0 and len(gws) == 0:
        self._fail_execution("pipeline 为空")
        return

    # 5. 创建自动重试策略
    AutoRetryStrategyCreator(execution, pipeline_id).batch_create_strategy(frozen_tree)

    # 6. 创建超时配置
    batch_create_timeout_configs(execution, frozen_tree)

    # 7. 保存 context (含 bamboo_pipeline_id + id_map)
    self.execution.context["bamboo_pipeline_id"] = bamboo_pipeline_id
    self.execution.context["bamboo_pipeline"] = pipeline
    self.execution.context["node_id_map"] = id_map

    # 8. 清理重新执行时可能残留的旧 pipeline data
    self._cleanup_pipeline_data(pipeline)

    # 9. 运行 pipeline (异步调度节点到 er_execute/er_schedule 队列)
    result = pipeline_api.run_pipeline(runtime=self._runtime, pipeline=pipeline)

    # 10. 处理验证错误 / 成功
    if not result.result:
        self._fail_execution(detail_msg, do_rollback=True)

```

1.4 Pipeline 重试清理

`[_cleanup_pipeline_data()]` 处理退出重试场景：同一 `[FlowExecution]` 重新执行时，旧的 `[pipeline.eri.models]` 数据会导致唯一约束冲突。该方法在 `[run_pipeline()]` 前删除所有与当前节点 ID 关联的记录：

```
def _cleanup_pipeline_data(self, pipeline):
    all_node_ids = {所有活动、网关、start/end 的 ID}
    PipelineData.objects.filter(node_id__in=all_node_ids).delete()
    PipelineState.objects.filter(node_id__in=all_node_ids).delete()
    PipelineExecutionData.objects.filter(node_id__in=all_node_ids).delete()
    PipelineSchedule.objects.filter(node_id__in=all_node_ids).delete()
    PipelineNode.objects.filter(node_id__in=all_node_ids).delete()
    PipelineExecutionHistory.objects.filter(node_id__in=all_node_ids).delete()
    PipelineCallbackData.objects.filter(node_id__in=all_node_ids).delete()
```

2. Pipeline Builder

位于 [opsflow/core/pipeline_builder/__init__.py]，将自定义 [{nodes, edges}] 格式的 [pipeline_tree] 转换为 [bamboo-engine] 可执行的 Pipeline Tree。

2.1 build_bamboo_pipeline() 流程

```
def build_bamboo_pipeline(flow_template, pipeline_tree=None, target_hosts=None,
                          global_vars=None, execution_id=None, excluded_nodes=None):
```

****Step-by-step 执行流程：****

```
Step 1: _filter_nodes_and_edges()
├── 排除 excluded_nodes 列表中的节点
├── 提取 visual_start_id / visual_end_id (前端 X6 的 start/end 节点)
└── 返回 effective_nodes / effective_edges (去掉 start_event/end_event 后的有效节点)

Step 2: 空节点检测
├── if not effective_nodes → 返回空 pipeline (只有 start → end)

Step 3: _parse_edge_conditions()
├── 扫描 edges, 解析条件表达式 + 自动变量 (_result_n1 等)

Step 4: _build_adjacency_lists()
├── 构建 out_edges / in_edges 邻接表

Step 5: _create_all_elements()
├── 遍历每个节点, 根据 node_type 创建对应 bamboo-engine builder 元素
│   ├── "" / "atom" → ServiceActivity(component_code="opsflow_plugin")
│   ├── "exclusive_gateway" → ExclusiveGateway()
│   ├── "parallel_gateway" → ParallelGateway()
│   ├── "conditional_para" → ConditionalParallelGateway()
│   ├── "converge_gateway" → ConvergeGateway()
│   ├── "approval" → ServiceActivity(atom_type="approval")
│   ├── "subprocess" → SubProcess(embedded) 或 ServiceActivity(independent)
│   └── 其他 → ServiceActivity(atom)

Step 6: _topological_connect()
├── 从入度为 0 的节点开始 BFS 拓扑遍历
├── 单出边 → 直接 connect()
├── 多出边 (success/failure) → 自动插入 ExclusiveGateway
├── 多出边 (其他标签) → 插入 ParallelGateway
└── 多根节点 → 自动插入 ParallelGateway 分叉

Step 7: _pair_converge_gateways()
├── BFS 搜索 parallel_gateway → converge_gateway 配对

Step 8: _build_data_inputs()
├── 注入 target_hosts / global_vars / execution_id
├── 注入项目环境变量 (ProjectEnvironmentVariable)
└── 注入自动变量 (NodeOutput 引用)

Step 9: _apply_timeout_configs()
├── 设置 pipeline.contrib.node_timeout 超时配置

Step 10: _build_id_map()
├── bamboo UUID → 前端原始节点 ID 映射
```

2.2 节点元素创建规则 (elements.py)

```
def _create_element(node, outgoing_edges, edge_conditions):
```

node_type	创建的元素	关键行为
`exclusive_gateway`	`ExclusiveGateway(id=nid)`	为每条出边添加条件
`parallel_gateway`	`ParallelGateway(id=nid)`	无条件并行
`conditional_parallel_gateway`	`ConditionalParallelGateway(id=nid)`	每条出边带条件
`converge_gateway`	`ConvergeGateway(id=nid)`	汇聚点
`approval`	`ServiceActivity(opsflow_plugin)`	`_atom_type='approval'`

node_type	创建的元素	关键行为
`subprocess` (独立)	`ServiceActivity(opsflow_plugin)`	`_atom_type='subprocess_independent'`
`subprocess` (嵌入)	`SubProcess(start, data, params)`	嵌入子流程 DAG
默认 (原子)	`ServiceActivity(opsflow_plugin)`	`_atom_type` 从 node 获取

2.3 网关配对逻辑

`_find_converge()` 使用 BFS 从 `[parallel_gateway]` 向下游搜索对应的 `[converge_gateway]`:

```
def _find_converge(gw_id, out_edges, effective_nodes):
    visited = {gw_id}
    q = deque()
    for e in out_edges.get(gw_id, []):
        q.append(e['to'])
    while q:
        nid = q.popleft()
        if nid in visited: continue
        visited.add(nid)
        node = next((n for n in effective_nodes if n['id'] == nid), None)
        if node.get('node_type') == 'converge_gateway':
            return nid
        for e in out_edges.get(nid, []):
            if e['to'] not in visited:
                q.append(e['to'])
    return None
```

3. PluginService 路由

位于 `[opsflow/core/plugin_service_adapter.py]`。替换了旧的 `[atom_service.py]` (按原子类型动态创建 `N` 个 `Component`)，改为单一 `[PluginService]` + `[OpsflowPluginComponent]`。

3.1 Component 注册

```
class OpsflowPluginComponent(Component):
    name = "OpsFlow Plugin"
    code = "opsflow_plugin"
    bound_service = PluginService
```

3.2 execute() 分发流程

```

PluginService.execute(data, parent_data)
|
|—— 1. _extract_params(data)
|    从 data.inputs 提取 _atom_type / _plugin_version / 用户参数
|
|—— 2. 特殊原子判断
|    |—— “subprocess_independent” → _execute_independent_subprocess()
|    |—— 其他 → 继续
|
|—— 3. get_plugin(atom_type, version) → PLUGIN_REGISTRY 查找
|
|—— 4. 变量解析 resolve_params(params, resolve_ctx, var_types)
|    从 parent_data 读取 global_vars / target_hosts
|    对参数做 ${key} 模板替换
|
|—— 5. plugin_instance.execute(**resolved_params)
|
|—— 6. 将结果写入 data.outputs
|    {_result, stdout, stderr, executor_output}
|
|—— 7. _promote_result(success, parent_data)
|    将 _result 立即提升到 pipeline 上下文（供排他网关条件评估）

```

3.3 schedule() 轮询

适用于 [_need_schedule=True] 的异步插件：

```

def schedule(self, data, parent_data, callback_data=None):
    # 同步插件 → 直接返回 True
    if not plugin_cls.need_schedule():
        return True
    # 异步插件 → 调用 schedule() 轮询
    result = instance.schedule(context=context, **params)
    if result is None: return None    # 继续等待
    return bool(result)              # True=完成, False=失败

```

3.4 rollback() 回滚

```

def rollback(self, data, parent_data):
    instance = plugin_cls()
    result = instance.rollback(context=context, **params)
    return result.get('success', False)

```

4. 信号系统

位于 [opsflow/signals/]，基于 [pipeline.eri.signals.post_set_state]。

4.1 信号处理链



4.2 原子节点状态更新 (state.py)

使用 [JSON_SET] 原子更新避免并发丢失:

```
def _update_execution_node_status(execution, node_id, to_state):
    node_state = map_bamboo_node_state(to_state)
    original_id = id_map.get(node_id, node_id) # 映射回原始 ID
    with connection.cursor() as cursor:
        cursor.execute(
            "UPDATE ops_flow_execution SET node_status = "
            "JSON_SET(COALESCE(node_status, '{}'), %s, CAST(%s AS JSON)) "
            "WHERE id = %s",
            [f'${original_id}', mapped, execution.id]
        )
```

回退方案 (非 MySQL 数据库): [refresh_from_db()] + read-modify-write。

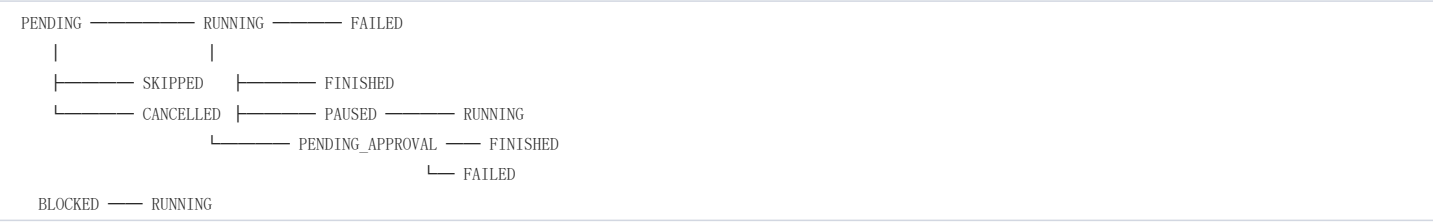
5. 状态机

5.1 NodeState (节点状态)

枚举值	含义	bamboo-engine 映射
`PENDING`	等待执行	READY
`RUNNING`	执行中	RUNNING

枚举值	含义	bamboo-engine 映射
`FINISHED`	成功完成	FINISHED
`FAILED`	执行失败	FAILED
`PAUSED`	已暂停	SUSPENDED
`SKIPPED`	已跳过	—
`CANCELLED`	已取消	REVOKED
`BLOCKED`	被阻塞	BLOCKED
`PENDING_APPROVAL`	等待审批	—

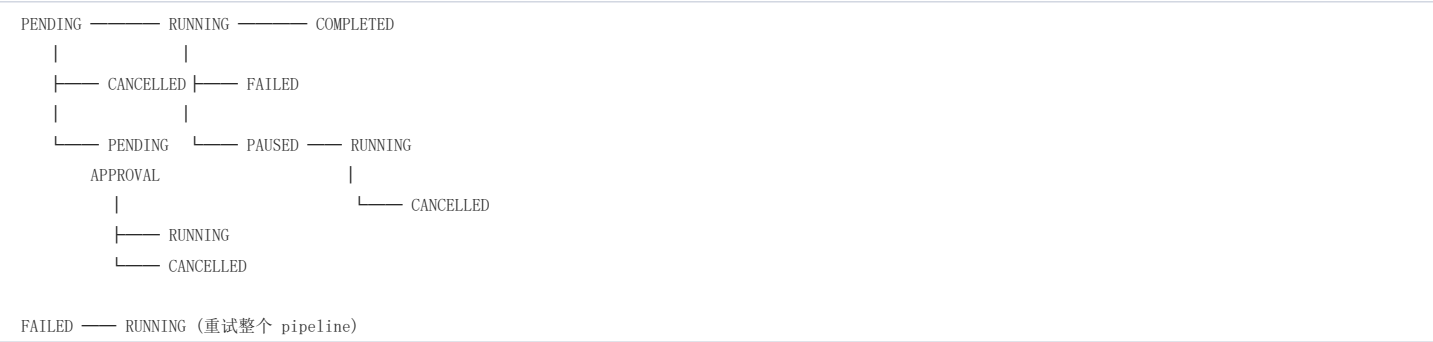
****节点状态流转矩阵：****



5.2 PipelineState (Pipeline 级状态)

枚举值	含义
`PENDING`	等待执行
`PENDING_APPROVAL`	等待审批
`RUNNING`	执行中
`PAUSED`	已暂停
`COMPLETED`	已完成
`FAILED`	已失败
`CANCELLED`	已取消

****Pipeline 级状态流转矩阵：****



5.3 状态校验函数

```
def validate_node_transition(current: NodeState, target: NodeState) -> bool
def validate_pipeline_transition(current: PipelineState, target: PipelineState) -> bool
```

API 层在 [ExecutionLifecycleMixin] 中使用 [validate_pipeline_transition()] 防止非法操作：

```
@action(detail=True, methods=['post'])
def start(self, request, pk=None):
    execution = self.get_object()
    if not validate_pipeline_transition(execution.status, PipelineState.RUNNING):
        return api_error(ErrorCodes.INVALID_STATE, msg=f"不能启动（当前：{execution.status}）")
```

6. 执行控制

6.1 节点操作（NodeCommandDispatcher）

位于 [opsflow/core/node_dispatcher.py]，封装所有节点级操作：

方法	操作	追踪记录
`retry(nid, operator)`	递增 retry_count → 创建 Trace → 调用 engine.retry()	NodeExecutionTrace + log_retry
`skip(nid, operator)`	记录操作人 → 调用 engine.skip()	context._last_operator
`force_fail(nid, operator, reason)`	记录操作人 → 调用 engine.force_fail()	ex_data 含 reason

API 端点通过 [ExecutionNodeCommandMixin] 暴露：[retry_node] / [skip_node] / [force_fail] / [batch_retry] / [batch_skip] / [retry_subprocess]。

批量操作逻辑：从 [execution.node_status] 过滤出 [status == 'failed'] 的节点列表，逐一执行命令。

6.2 审批流程（ExecutionApprovalMixin）

```
User 提交执行 → 执行到 approval 节点 → PluginService 回调 → status PAUSED
→ 前端获知 pending_approval → 审批人 approve/reject
├── approve → _record_approval_decision() → FlowEngine.resume()
└── reject → _record_approval_decision() → FlowEngine.resume() → 继续失败路径
```

审批决策存储在 [execution.context._approval_decisions] 中，包含 [by] (审批人)、[at] (时间)、[approved] (同意/拒绝) 等字段。

7. 自动重试机制

```
FlowEngine.run() → AutoRetryStrategyCreator.batch_create_strategy(pipeline_tree)
├── 遍历 nodes, 为 max_retries > 0 的节点创建 AutoRetryStrategy 记录

Node FAILED 信号 → dispatch_auto_retry(execution, node_id)
├── AutoRetryStrategy 检查 retry_times < max_retry_times?
├── 否 → 返回 False (进入正常失败处理)
└── 是 → 派发 auto_retry_node_task (countdown=interval) → FlowEngine.retry()
```

AutoRetryStrategy 字段	说明
`execution` / `node_id`	执行 + 节点标识
`max_retry_times` (max 10)	最大重试次数
`interval` (max 3600s)	重试间隔
`retry_times`	当前已重试计数器

8. 超时机制

```
FlowEngine.run() → batch_create_timeout_configs(pipeline_tree)
└── 为 timeout_seconds > 0 的节点创建 NodeTimeoutConfig

RUNNING 信号 → Redis ZADD opsflow:executing_nodes {key: deadline}
FINISHED/FAILED → Redis ZREM
APScheduler 10s轮询 → dispatch_timeout_nodes()
└── ZRANGEBYSCORE 到期节点 → execute_node_timeout_strategy Celery 任务
```

超时策略	Handler	行为
`forced_fail`	`ForcedFailStrategy`	调用 `api.forced_fail_activity()` 强制失败
`forced_fail_and_skip`	`ForcedFailAndSkipStrategy`	先 force_fail 再 skip

Redis 键 [opsflow:executing_nodes] (Sorted Set)，member=[{node_id}_{version}]，score=当前时间+timeout_seconds。

9. 轨迹追踪

[NodeExecutionTrace] 模型记录每个节点每次执行的完整历史：

关键字段	说明
`execution` / `node_id`	执行 + 节点标识，unique_together=(exec, nid, retry_count)
`status` / `status_history`	当前状态 + 变更历史
`entered_at/exited_at/duration_ms`	进入/退出时间 + 耗时 (ms)
`inputs/outputs/error`	输入输出快照 + 错误信息
`retry_count` / `log_file_path`	重试次数 + 日志文件路径

JSON Lines 日志（[{LOG_DIR}/opsflow/tasks/{id}/{node}.log]）：事件类型 [state|output|error|tower|retry]，每行结构化 JSON。

10. Celery 任务

任务	队列	说明
`execute_pipeline_task`	`er_execute`	主执行任务 (max_retries=3)
`auto_retry_node_task`	默认	自动重试 (countdown=interval)
`execute_node_timeout_strategy`	默认	超时策略执行
`webhook_send`	默认	Webhook HTTP 回调 + 重试

11. WebSocket 推送

信号处理器调用 [channel_layer.group_send("execution_{id}", {"type": "execution.completed", "status": status})] 推送状态变更，同时实时推送节点级状态（用于前端 MonitorCanvas 实时着色更新）。